

Clothes Segmentation on ATR - Code Documentation

This is the code manual, not the assignment report. The report (dataset choice, model, loss, results, strengths / weaknesses and limitations) is `reports/REPORT.pdf` (Markdown: `reports/REPORT.md`).

This document explains the repository itself: the problem framing, the data pipeline, the design decisions, every module, and how to run and extend the system. For the results see `reports/REPORT.md`; every number there comes from the run stored in `outputs_15epochs/`.

1. Problem statement

Given a photograph of a person, label every pixel with the garment (or body part / background) it belongs to, so that the clothes can be cut out, replaced or measured - the core operation of a virtual fitting room. The assignment (task code CVCY002) asks for:

1. dataset selection, preprocessing and augmentation;
2. a trained deep-learning segmentation model and a pipeline that takes a personal image and returns the segmented clothes;
3. an analysis of strengths, weaknesses and operating limits;
4. deliverables: documented script, report, README in a GitHub repository.

We treat the task as **18-class semantic segmentation** (the ATR label set). A binary "clothes / not clothes" mask is derived from the 18-class prediction by taking the union of the 11 clothing classes, so the model serves both the coarse (cut-out) and fine (per-garment) use cases.

2. Dataset

2.1 Choice

`mattdmjaga/human_parsing_dataset` on the Hugging Face Hub is the **ATR** human-parsing dataset (Liang et al., *Deep Human Parsing with Active Template Regression*, 2015) packaged as two parquet shards (about 0.8 GB, 17,706 rows, columns `image` and `mask`). Every mask is a single-channel PNG whose pixel value is the class id:

id	class	group	id	class	group
0	Background	background	9	Left-shoe	clothes
1	Hat	clothes	10	Right-shoe	clothes
2	Hair	body	11	Face	body
3	Sunglasses	clothes	12	Left-leg	body
4	Upper-clothes	clothes	13	Right-leg	body
5	Skirt	clothes	14	Left-arm	body
6	Pants	clothes	15	Right-arm	body
7	Dress	clothes	16	Bag	clothes
8	Belt	clothes	17	Scarf	clothes

Reasons for choosing it over alternatives (LIP, CIHP, DeepFashion2, iMaterialist, Clothing Co-Parsing):

- garment-level classes that map directly onto fitting-room needs (tops, dresses, skirts, pants, shoes, accessories);
- single-person, full-body photos - the same input distribution as the target application;
- small enough to download and train on a free cloud GPU in about an hour, large enough for a transformer to reach a strong score;
- zero-friction loading (`datasets.load_dataset`), no registration, no manual archive handling;
- a public reference model trained on the same data (`mattmdjaga/segformer_b2_clothes`) exists, which lets us sanity-check our numbers.

Known caveats: only one person per image, mostly frontal fashion photos, some label noise around hair / face and left / right assignments, and several near-synonymous classes (dress vs. upper-clothes + skirt).

2.2 Split

The Hub dataset ships as a single train split. `data.make_splits` draws a random permutation with `seed=42` and assigns **5 % test, 5 % validation, 90 % train** (885 / 885 / 15,936 images). The indices are written to `outputs/data/split.json` on first use and re-read afterwards, so `train`, `evaluate`, `baseline` and `predict` `--from-test` always see the same images. `--limit N` (smoke tests) truncates the lists after loading and never changes the stored file.

2.3 Preprocessing

- RGB conversion, resize to `img_size x img_size` (default 512) with bilinear interpolation for images and nearest-neighbour for masks;
- ImageNet mean / std normalisation (all SegFormer checkpoints were trained with it);
- labels are kept as `int64` class ids; 255 marks pixels to ignore.

Evaluation up-samples the logits to the original mask size before the arg-max, so the reported metrics are at native resolution and not flattered by down-sampled labels.

2.4 Augmentation (training split only)

transform	parameters	why
Horizontal flip + label swap	<code>p = 0.5</code> ; swaps (9,10), (12,13), (14,15)	doubles pose variety; the swap keeps left / right semantics correct
Affine	scale 0.75-1.25, translate up to 5 %, rotate up to 12 deg, <code>p = 0.7</code> ; padding -> image 0, mask 255	camera distance, framing and tilt variation
RandomBrightnessContrast	<code>0.2 / 0.2</code> , <code>p = 0.5</code>	exposure differences
HueSaturationValue	<code>10 / 20 / 10</code> , <code>p = 0.4</code>	white balance and fabric colour variety
GaussianBlur	kernel 3-5, <code>p = 0.1</code>	soft-focus phone photos

Augmentations are implemented with **Albumentations**, which applies identical geometric transforms to the image and the mask. The flip is done manually in `ATRDataset.__getitem__` because the label swap must know whether the flip happened. `python run.py prepare` writes `outputs/data/augmentation_preview.png` so the effect can be inspected.

3. Model

3.1 SegFormer-B2

SegFormer (Xie et al., NeurIPS 2021) couples a hierarchical Transformer encoder with a very light MLP decoder.

Encoder (MiT-B2, ~24.7 M parameters)

stage	stride	channels	layers	attention reduction
1	4	64	3	8
2	8	128	4	4
3	16	320	6	2
4	32	512	3	1

- *Overlapping patch embedding* (conv 7x7 stride 4, then 3x3 stride 2) keeps neighbouring patches connected.
- *Efficient self-attention* shrinks the key / value sequence by the reduction ratio, giving global context at 512x512 for a manageable cost.
- *Mix-FFN* inserts a 3x3 depth-wise convolution into the feed-forward block, which supplies position information and makes the network resolution-agnostic (no positional embeddings to interpolate).

Decoder (all-MLP, ~3 M parameters) - each stage output is linearly projected to 768 channels, up-sampled to stride 4, concatenated, fused with a linear layer, and classified by a 1x1 convolution into 18 logits at 1/4 resolution. We bilinearly up-sample the logits to the label size inside the loss and at inference.

Total: ~27.4 M parameters, 4-5x smaller than a DeepLabV3+ ResNet-101 with better ADE20K accuracy.

3.2 Initialisation

nvidia/segformer-b2-finetuned-ade-512-512: encoder pretrained on ImageNet-1k, encoder + decoder trained on ADE20K (150 scene classes, including *person*). Transformers' `ignore_mismatched_sizes=True` drops the 150-way classifier and creates a fresh 18-way one. Alternatives selectable with `--pretrained`:

- `nvidia/mit-b2` - ImageNet-only encoder, decoder from scratch (a little slower to converge);
- `nvidia/segformer-b0-...` / `b5-...` - smaller or larger variants;
- `mattmdjaga/segformer_b2_clothes` - already fine-tuned on ATR (useful only for a warm start; it is also the model evaluated by `python run.py baseline`).

3.3 Why not another architecture?

- **U-Net / DeepLab (CNN)**: strong on local texture, but garments require whole-body context (is this a dress or a top and a skirt?). Attention gives that context at every layer.
- **Mask2Former / SAM**: more accurate on some benchmarks but 3-10x heavier, harder to fine-tune on one GPU, and SAM has no class vocabulary. SegFormer-B2 hits the accuracy / cost / simplicity sweet spot for this task.

4. Loss function

`losses.SegmentationLoss = CrossEntropy(ignore 255) + dice_weight x SoftDice`, with `dice_weight = 0.5`.

- Cross-entropy: fast, stable convergence, well-calibrated probabilities; but it averages over pixels, so it is dominated by background / upper-clothes / pants (together > 80 % of pixels) and barely notices belts, sunglasses or scarves.
- Soft Dice (per class, averaged over classes): $1 - \frac{\text{mean}_c (2 * \sum p_c * y_c + 1)}{(\sum p_c + \sum y_c + 1)}$. Because each class is normalised by its own size, a 0.3 % belt has the same weight as the background, and the loss directly optimises the overlap measure (IoU / Dice) we report.

- Combining them keeps CE's smooth gradients while Dice corrects the imbalance. Class-weighted CE was rejected because it needs hand-tuned weights and over-predicts rare classes; focal loss addresses hard pixels rather than small regions; Lovasz-softmax is slower and gave no gain in short trials.
- Pixels labelled 255 (affine padding) are excluded from both terms and from the metrics.

5. Training procedure

Implemented in `train.py` with plain PyTorch:

item	value
optimiser	AdamW, lr 6e-5, weight decay 0.01, betas (0.9, 0.999)
schedule	linear warm-up 300 steps, then polynomial (power 1) decay to 0
batch size	8 (512x512); 4 recommended on GPUs with <= 16 GB
epochs	10 (about 20k steps)
precision	bf16 autocast on Ampere+ GPUs, fp16 + GradScaler on older GPUs (T4), fp32 on CPU
gradient clipping	global norm 1.0
model selection	highest validation mean IoU
seed	42 (Python, NumPy, PyTorch)

Hardware used for the reported run: a single NVIDIA GeForce RTX 4060 Laptop GPU (8 GB) on Windows 11, bf16 mixed precision, batch 4, `--num-workers 0`; 15 epochs took 4.4 h (about 17.5 min per epoch, 3,984 steps per epoch). No cloud GPU was used. `train_summary.json` records the device, GPU name and precision of every run.

Per epoch the loop trains, validates (`evaluate.run_validation`), appends a row to `outputs/history.csv` (train / val loss, val mIoU, pixel accuracy, mean accuracy, Dice, clothes IoU, learning rate, epoch time), saves `checkpoints/last` (+ `training_state.pt` for resuming) and, when validation mIoU improves, `checkpoints/best` with its metrics. Every `--log-every` steps the current loss is appended to `outputs/train_steps.csv` so the report can plot a fine-grained loss curve. Checkpoints are standard Hugging Face folders (`config.json`, `model.safetensors`, `preprocessor_config.json`) that load with `SegformerForSemanticSegmentation.from_pretrained`.

`--resume` reloads `checkpoints/last`, the optimiser / scheduler state and the history and continues from the next epoch.

6. Evaluation

`evaluate.evaluate` runs the best checkpoint over the test split, batch by batch:

1. forward pass at network resolution (loss and a reference mIoU are computed here);
2. for every image, up-sample the logits to the **original** mask size, arg-max, and accumulate an 18x18 confusion matrix;
3. derive metrics from the matrix (`metrics.SegMetrics`).

metric	definition	purpose
mean IoU	mean over classes of $TP / (TP + FP + FN)$	primary semantic-segmentation score
foreground mean IoU	same without Background	removes the trivially high background class
pixel accuracy	correct pixels / all pixels	overall, background-dominated
mean class accuracy	mean over classes of $TP / (TP + FN)$	recall per class
mean Dice	mean over classes of $2TP / (2TP + FP + FN)$	overlap score matched to the Dice loss
clothes-binary IoU / Dice	union of the 11 clothing classes vs. everything else	the "cut the clothes out" metric
per-class IoU / accuracy / Dice / support	per row of the matrix	strengths & weaknesses analysis
images per second	forward-pass throughput	deployment feasibility

Outputs: `outputs/eval/test_metrics.json` (everything, including the confusion matrix), `outputs/eval/test_per_class.csv`, `outputs/eval/test_qualitative.png`. `python run.py baseline` writes the same files with the `baseline_` prefix for the public `mattmdjaga/segformer_b2_clothes` model, evaluated with exactly the same protocol. Note that this model was trained on the whole ATR set and has most likely seen our test images, so it is an optimistic reference.

7. Inference pipeline

`predict.ClothesSegmenter` is the deployable component:

```
photo (any size) -> RGB -> resize 512x512 -> normalise -> SegFormer -> logits (1/4 res)
-> bilinear up-sample to photo size -> arg-max -> 18-class label map
-> colour mask | overlay | clothes-only RGBA cut-out | JSON summary
```

- `predict(image)` returns the (H, w) uint8 label map at the photo's own resolution.
- `segment_to_files(image, out_dir, stem)` writes `<stem>_mask.png`, `<stem>_overlay.jpg`, `<stem>_clothes.png` (alpha = 255 on clothing classes only) and `<stem>_labels.json` with the classes present and their pixel percentage plus the total clothing coverage.
- `--tta` enables horizontal-flip test-time augmentation: the image and its mirror are both predicted, the mirrored softmax is flipped back with the left / right class channels swapped, and the two probability maps are averaged (`model.forward_logits`). No retraining; +0.5 mIoU on the test split; throughput drops from ~53 to ~12 img/s on the RTX 4060 laptop GPU.
- `python run.py predict --input <file|folder>` batch-processes user images; `--from-test N` runs the first N test images and additionally saves the input and ground truth for side-by-side inspection (the report uses these for its prediction gallery).

Typical latency: 20-40 ms per 512x512 forward pass on a T4 (fp16), ~0.5 s on a laptop CPU.

8. Code map

file	responsibility
run.py	argument parsing, sub-commands, all orchestration
src/clothseg/config.py	dataset / model ids, class names, clothing ids, flip pairs, palette, Config dataclass
src/clothseg/data.py	load_raw_dataset, make_splits, swap_left_right, build_transforms, ATRDataset, make_loader, denormalize
src/clothseg/model.py	build_model, load_model, save_model, count_parameters, pick_device, autocast_dtype
src/clothseg/losses.py	soft_dice_loss, SegmentationLoss, upsample_logits
src/clothseg/metrics.py	SegMetrics (confusion-matrix metrics), table / summary formatting
src/clothseg/prepare.py	download, split, statistics, class-frequency plot, augmentation preview
src/clothseg/train.py	training loop, scheduler, checkpointing, CSV / JSON logging, resume
src/clothseg/evaluate.py	run_validation, evaluate, evaluate_baseline
src/clothseg/predict.py	ClothesSegmenter, run_predict
src/clothseg/visualize.py	colorize, overlay, clothes_cutout, save_grid

All modules are importable from Python (`sys.path.insert(0, "src")`) for notebooks or serving code.

9. Running on a cloud GPU (optional, step by step)

The reported results come from a local laptop GPU; this section only matters if no local GPU is available. The steps are written for Lightning AI but apply to any cloud notebook with a terminal.

1. Sign in at lightning.ai, create a **Studio**, and switch the machine to a GPU (T4 for smoke tests; L4 / A10G / A100 for the full run).
2. Terminal: `bash git clone <repo-url> cd clothes-segmentation-atr pip install -r requirements.txt`
`export HF_HOME=/teamspace/studios/this_studio/hf_cache # keep the dataset in persistent storage`
`python run.py check`
3. Smoke test first (3 minutes): `python run.py all --limit 400 --epochs 1 --img-size 384 --skip-baseline`
4. Full run in the background: `nohup python run.py all > run.log 2>&1 &` and follow with `tail -f run.log`.
Expected duration: about 1.5-2 h on a T4, 45 min on an L4, 25 min on an A100.
5. Inspect `reports/REPORT.md`, then `git add` the report, docs and the small output files listed in the README and push. Weights (outputs/checkpoints/best, ~110 MB) can be uploaded to the Hugging Face Hub with `model.push_to_hub(...)` or attached to a GitHub release.

Out-of-memory: use `--batch-size 4` (or `--img-size 384`). Interrupted: `python run.py train --resume` then `python run.py evaluate`.

Windows note: `--num-workers` defaults to 0 on Windows because every DataLoader worker process re-imports torch + CUDA (about 2 GB of committed memory each) and can exhaust the page file; on Linux the default is 4.

10. Extending the system

- **Binary clothes model:** map the 11 clothing ids to 1 and the rest to 0 in `ATRDataset.__getitem__` and set `NUM_CLASSES = 2`; everything else is unchanged.
 - **More data:** concatenate another Hugging Face dataset with the same label ids in `load_raw_dataset`.
 - **Bigger model:** `--pretrained nvidia/segformer-b5-finetuned-ade-640-640 --img-size 640 --batch-size 4`.
 - **Multi-scale test-time augmentation:** horizontal-flip TTA already exists (`model.forward_logits(..., tta=True)`), enabled with `--tta`; +0.5 mIoU on the test split). Extending it with 0.75x / 1.25x rescaled passes is a small change in the same function.
 - **Deployment:** export with `torch.onnx.export(model, pixel_values, ...)`; the decoder has no dynamic ops.
-

11. Reproducibility checklist

- deterministic split stored in `outputs/data/split.json` (seed 42);
- run configuration in `outputs/run_config.json`;
- metrics in `outputs/eval/*.json` and `*.csv`; history in `outputs/history.csv`;
- checkpoint in `outputs/checkpoints/best` (Hugging Face format);

Residual non-determinism comes from cuDNN kernels and multi-worker data loading; re-runs differ by a few tenths of a mIoU point.